CodeS: Towards Building Open-source Language Models for Text-to-SQL

Haoyang Li
Renmin University of China, China
lihaoyang.cs@ruc.edu.cn

Jing Zhang*
Renmin University of China, China
zhang-jing@ruc.edu.cn

Hanbing Liu
Renmin University of China, China
liuhanbing@ruc.edu.cn

Ju Fan
Renmin University of China, China
fanj@ruc.edu.cn

Xiaokang Zhang
Renmin University of China, China
zhang2718@ruc.edu.cn

Jun Zhu
AI-Finance, China
zhujun@ai-finance.cn

Renjie Wei
AI-Finance, China
weirenjie@ai-finance.cn

Hongyan Pan
AI-Finance, China
panhongyan@ai-finance.cn

Cuiping Li
Renmin University of China, China
licuiping@ruc.edu.cn

Hong Chen
Renmin University of China, China
chong@ruc.edu.cn

arXiv:2402.16347v1 [cs.CL] 26 Feb 2024

ABSTRACT

Language models have shown promising performance on the task of translating natural language questions into SQL queries (Text-to-SQL). However, most of the state-of-the-art (SOTA) approaches rely on powerful yet closed-source large language models (LLMs), such as ChatGPT and GPT-4, which may have the limitations of unclear model architectures, data privacy risks, and expensive inference overheads. To address the limitations, we introduce CodeS, a series of pre-trained language models with parameters ranging from 1B to 15B, specifically designed for the text-to-SQL task. CodeS is a fully open-source language model, which achieves superior accuracy with much smaller parameter sizes. This paper studies the research challenges in building CodeS. To enhance the SQL generation abilities of CodeS, we adopt an incremental pre-training approach using a specifically curated SQL-centric corpus. Based on this, we address the challenges of schema linking and rapid domain adaptation through strategic prompt construction and a bi-directional data augmentation technique. We conduct comprehensive evaluations on multiple datasets, including the widely used Spider benchmark, the newly released BIRD benchmark, robustness-diagnostic benchmarks such as Spider-DK, Spider-Syn, Spider-Realistic, and Dr.Spider, as well as two real-world datasets created for financial and academic applications. The experimental results show that our CodeS achieves new SOTA accuracy and robustness on nearly all challenging text-to-SQL benchmarks.

*Jing Zhang is the corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY
© 2018 Association for Computing Machinery.
ACM ISBN 978-1-4503-XXXX-X/18/06…$15.00
https://doi.org/XXXXXXX.XXXXXXX

KEYWORDS

Text-to-SQL, Large Language Model

ACM Reference Format:
Haoyang Li, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, and Hong Chen. 2018. CodeS: Towards Building Open-source Language Models for Text-to-SQL. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation emai (Conference acronym 'XX)*. ACM, New York, NY, USA, 16 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

The text-to-SQL task involves translating a user's natural language (NL) question into a corresponding and valid Structured Query Language (SQL) query that can be executed over a database. Figure 2 illustrates how an NL question posed over a database (*e.g.*, Bank Financial) can be translated into an SQL query. Text-to-SQL enables users who may not be familiar with SQL or the structure of a database to interact with the database using natural language, and thus it has garnered increasing attention from both the database [4, 18, 24] and natural language processing communities [36, 57].

State-of-the-Art: Strengths and Limitations. While traditional text-to-SQL utilizes the supervised fine-tuning approach (SFT) [36, 57, 67], more recently, the paradigm has started to shift with the advent of large language models (LLMs) like GPT-4 [48], GPT-3.5 [49], and PaLM-2 [1]. Instead of relying heavily on fine-tuning, LLMs have shown their capability in text-to-SQL using thoughtfully crafted prompts [9, 16, 40, 51, 55, 61], which is known as "prompt learning" or "in-context learning" [42].

However, most of the state-of-the-art (SOTA) approaches rely on *closed-source* LLMs, such as DIN-SQL [51] based on GPT-4, SQL-PaLM [61] based on PaLM-2 and C3 [16] built upon GPT-3.5. Although achieving promising text-to-SQL performance, these approaches may have the following limitations. (**L1**) Closed-source models hide their architectures and training/inference details, hindering purpose-specific continuous development tailored to specific applications. (**L2**) Invoking these models via APIs risks data privacy,

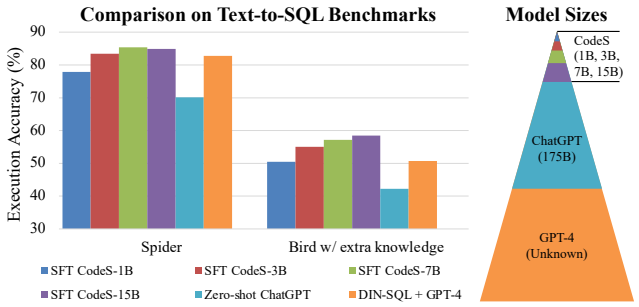


Figure 1: Comparisons between CodeS and SOTA LLMs on two challenging text-to-SQL benchmarks, Spider [79] and BIRD [38]. While 10x-100x smaller than the existing SOTA LLMs, CodeS achieves comparable or even superior accuracy.

as data must be sent to model providers. (**L3**) Most closed-source models have numerous parameters (e.g., 175B parameters for GPT-3.5), causing significant inference overheads, which are typically reflected by the monetary costs of calling APIs.

Our proposals. This paper introduces CodeS, an *open-source* language model that is designed for real-world text-to-SQL applications. CodeS is built upon StarCoder [39], a cutting-edge LLM designed specifically for code generation, with varying parameters ranging from 1B to 15B. Users can select an appropriately sized CodeS based on their computational resources to construct their text-to-SQL applications. As depicted in Figure 1, compared with the SOTA text-to-SQL solutions, CodeS has the following advantages.

- **Fully Open-source LLM.** CodeS, which is built upon StarCoder [39], is fully open-source and public to users.
- **New SOTA Results.** CodeS achieves the **new SOTA performance** on nearly all challenging text-to-SQL benchmarks, such as Spider [79] and BIRD [38].
- **Small Sizes.** CodeS is 10x-100x smaller than the existing SOTA LLMs, such as ChatGPT and GPT-4[1].

Challenges and Solutions. We outline the main technical challenges in developing CodeS and explain our solutions as follows.

C1: How to enable small-sized language models with complex text-to-SQL reasoning capacity? Directly using pre-trained language models, such as LLaMA-2 [65] and StarCoder [39], faces challenges in text-to-SQL, mainly because SQL-related content typically constitutes only a tiny fraction of their entire pre-training corpus. Subsequently, the data bias could potentially hinder the text-to-SQL capability, as language models during the pre-training phase aim to fit the distribution of the entire corpus, rather than just the distribution of SQL-related data. Moreover, compared with ChatGPT or GPT-4, the small-sized language models possess limited reasoning capacity, resulting in insufficient capabilities on text-to-SQL. To address the challenge, we propose an incremental pre-training approach utilizing a vast curated dataset relevant to the text-to-SQL task. Specifically, we collected 21.5GB of data, consisting of SQL-related data (11GB), NL-to-code data (6GB), and NL-related data (4.5GB), with the aim of enhancing both SQL generation capabilities and natural language comprehension. By incrementally pre-training StarCoder [39] on this corpus, we created a series of CodeS models, with varying parameters ranging from 1B to 15B.

C2: How to generate high-quality prompts for text-to-SQL to alleviate the difficulty of schema linking? Schema linking is crucial for text-to-SQL translation, ensuring models map input questions to specific database elements [33]. Yet, issues emerge with numerous tables, wide tables with numerous columns, ambiguous table/column names, and large tables with vast values. To address the challenge, we use a schema filtering strategy for numerous tables and wide tables, retaining only relevant tables and columns based on the user's query from the database, ensuring the schema doesn't exceed the model's context length. For ambiguous names, like abbreviations, we tie comments to these names, aiding models in understanding context. For large tables, a "coarse-to-fine" approach is adopted: we initially filter values using the BM25 index based on the question and further refine them with the longest common substring algorithm. Using these techniques, we frame prompts for the CodeS model, enhancing schema linking and boosting text-to-SQL performance for complex databases.

C3: How to adaptively transfer to databases within a new domain? In real-world applications, we aim for the CodeS model to adapt across various domains. A significant obstacle, however, is the lack of specific (question, SQL) pairs for fine-tuning. To address this challenge, we employ a bi-directional data augmentation technique. Firstly, we collect a few genuine user queries, manually annotate corresponding SQL queries, and leverage GPT-3.5 to produce a wider set of (question, SQL) pairs, ensuring user-oriented authenticity. On the other hand, we utilize SQL templates and their question templates from text-to-SQL benchmarks. By plugging in tables, columns, and values from the databases of new domains, we generate a diverse set of (question, SQL) pairs. This templating approach aids CodeS's adaptability to new distributions. In essence, our crafted training dataset combines real examples with structured templates, guaranteeing both authenticity and broad applicability.

Evaluation. We evaluate the created CodeS on two challenging text-to-SQL benchmarks: Spider [79] and BIRD [38]. While Spider has long been a standard for text-to-SQL, BIRD offers more complex questions, ambiguous schema, and large and dirty database values. The leading text-to-SQL method, DIN-SQL+GPT-4, manages only around 56% on its test set. Beyond the conventional Spider benchmark, we also assess CodeS against Spider's four distinct variants: Spider-DK [20], Spider-Syn [19], Spider-Realistic [12], and Dr.Spider [7]. These span a total of 20 test sets and are tailored to probe model resilience, especially in scenarios where test data distributions differ from training data distributions. To investigate the effect of our bi-directional data augmentation approach in rapidly adapting to new domains, we sourced databases from both the academic and finance domains. Given that both databases had insufficient training data for effective fine-tuning, we augmented the training data, fine-tuned our model, and then evaluated its performance on the respective test sets.

Our contributions are summarized as follows:

- We introduce CodeS, a series of language models ranging from 1B to 15B parameters, designed specifically for SQL

[1]It's worth noting that specific details regarding the scale of ChatGPT and GPT-4 have not been made public. Therefore, we follow the general assumption that ChatGPT is approximately equivalent in size to GPT-3-175B [32] and GPT-4 significantly exceeds the size of ChatGPT [47, 48].

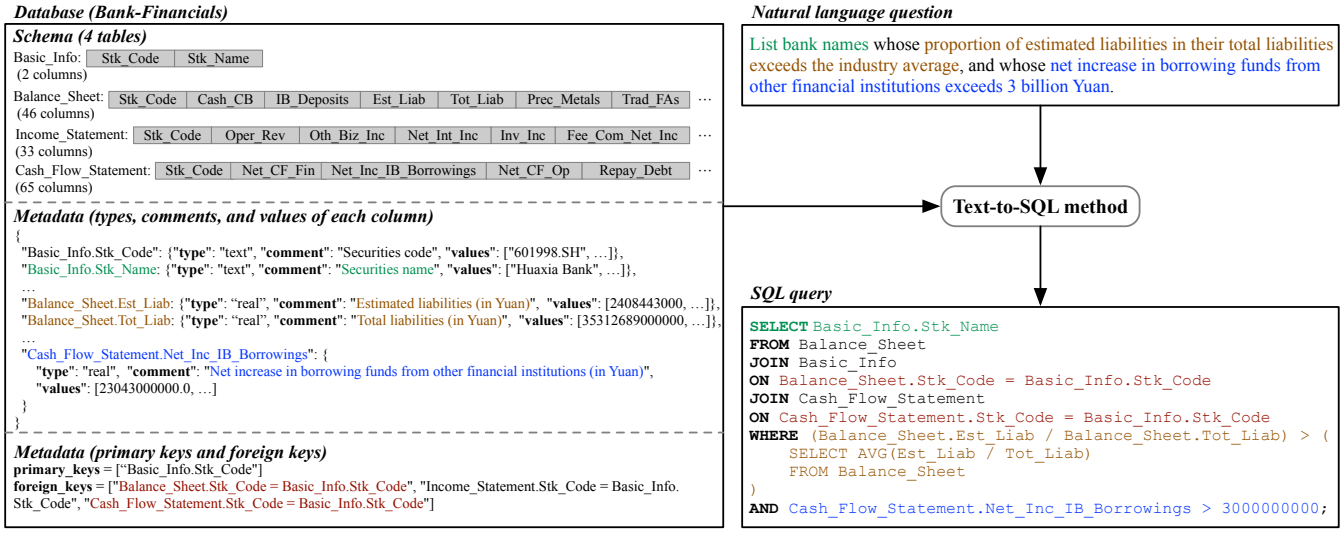


Figure 2: An example of text-to-SQL in the finance domain.

generation. This innovation is underpinned by an incremental pre-training technique and a meticulously curated pre-training corpus, comprising SQL-related, NL-related, and NL-to-code data. This approach marks a significant advancement in language models tailored for text-to-SQL applications.

- We enhance CodeS's performance using a comprehensive database prompt. Additionally, to facilitate its adaptation to new domains, we introduce a bi-directional data augmentation approach with limited annotation overhead.
- Through extensive evaluations on multiple text-to-SQL benchmarks, we demonstrate that (1) CodeS surpasses other notable open-source pre-trained language models in SQL generation capability; (2) When fine-tuned, CodeS achieves new SOTA accuracy and robustness on almost all challenging text-to-SQL benchmarks.
- We have open-sourced our code, models, and data on GitHub[2] to foster further research, adoption, and innovation in the text-to-SQL domain within the community.

2 RELATED WORK

In our survey, we cover supervised fine-tuning and prompting-based methods for text-to-SQL. Furthermore, we explore existing code language models because text-to-SQL can be viewed as a sub-task of code generation. Additionally, we examine various schema linking and data augmentation techniques that have been proposed to enhance text-to-SQL methodologies.

Supervised Fine-Tuning-Based Text-to-SQL. Before the era of LLMs, the mainstream approach in text-to-SQL is fine-tuning an "encoder-decoder" neural network model. Significant efforts have been made to enhance the representation capability of the encoder that encodes both the question and the database by incorporating graph structural information that exists between query tokens, tables, and columns using graph-relational neural networks [5, 67]. Some other efforts focus on injecting SQL grammar into the decoder, which constrains the output space of the decoder, ensuring the generation of syntactically correct SQL queries [18, 57, 75]. With the advancement of language models, there has been an increasing trend in formatting text-to-SQL as a sequence-to-sequence task [17, 24, 36, 37, 57, 58], where the input sequence consists of a natural language question and flattened database information including tables, columns, foreign keys, etc., and the output sequence is the target SQL query. Then, sequence-to-sequence language models such as T5 [53], and BART [34] are fine-tuned on these input-output sentence pairs, enabling them to generate SQL queries from the provided input. Inspired by the remarkable achievements of pre-training techniques [14, 52, 53], a series of studies [13, 27, 59, 76, 77] have explored pre-training language models using extensive database-related data and various pre-training objectives. However, in contrast to our CodeS, their primary goal isn't directly enhancing the SQL generation capability of language models. Instead, they focus on enhancing the encoder's ability to better represent the question and the database schema. Then, these pre-trained encoders are integrated into the "encoder-decoder" models.

Prompting-Based Text-to-SQL. The advent of LLMs, such as GPT-3 [3], Codex [8], PaLM [10], LLaMA [64], StarCoder [39], has brought about a revolutionary transformation in the field of NLP, achieving remarkable progress on various complex tasks that require reasoning abilities without fine-tuning any parameters [9, 40, 51]. For text-to-SQL, by leveraging a few text-to-SQL demonstrations as the few-shot prompt, SQL-PaLM [61] (based on PaLM-2) and Self-Debugging [9] (based on Codex) successfully achieve the state-of-the-art (SOTA) performance on Spider. In addition, inspired by chain-of-thought reasoning [70], designing effective prompts to stimulate the text-to-SQL capability of LLMs has become a hot research topic. DIN-SQL [51] (based on GPT-4) breaks down the text-to-SQL task into several simpler sub-tasks, including schema linking, query classification & decomposition, and SQL generation. C3 [16] (based on ChatGPT), by providing appropriate

[2]https://github.com/RUCKBReasoning/codes

instructions to ChatGPT, makes it become an experienced zero-shot text-to-SQL parser. Although these prompting-based methods have achieved SOTA performance on text-to-SQL benchmarks, as we analyzed in Section 1, it is challenging to implement them in real-world applications due to the significant costs and potential data privacy concerns associated with using these models' APIs.

Code Language Models. Over the past few years, there has been a growing interest in leveraging language models for coding-related tasks such as code understanding and generation [8, 39, 45, 46]. Existing code language models are often pre-trained on a diverse mix of programming languages, such as C, C++, Python, Java, C#, and SQL. This broad-spectrum training data can result in suboptimal performance for small-scale models on a specific programming language (*e.g.*, SQL) due to their constrained representation ability. To address this issue, we develop CodeS – a collection of open-source generative language models, uniquely optimized with an emphasis on a mixture of SQL-centric data.

Schema Linking. Schema linking plays a crucial role in text-to-SQL processes, aiming to identify referenced database schemas (such as tables and columns) and database values within natural language questions [33]. There are primarily two strategies for schema linking: string matching-based [4, 17, 26, 67] and neural network-based [2, 25, 36]. The string matching-based approach, simple yet effective, identifies schemas and values related to a question through direct string matching. However, this approach has limitations in certain scenarios, such as dealing with synonyms. To address these challenges, neural network-based methods have been developed. These methods are designed to assess the relevance of schemas and values at a semantic level. Once the schema linking results are obtained, for example, the matching degrees for all tables and columns, many techniques [4, 17, 67] incorporate these results as additional input for the text-to-SQL model. Different from them, RESDSQL [36] utilizes these results for schema pruning, retaining only the schemas most relevant to the input of the subsequent neural network, thus reducing the length of the input to the LLMs.

Data Augmentation for Text-to-SQL. Recent times have seen a growing interest in synthesizing data for text-to-SQL. The aim is to automatically generate (question, SQL) pairs that are relevant to a given database. Many current methods [29, 68, 71, 83] employ a SQL-to-question synthesis pipeline. This process typically involves first auto-generating SQL queries according to a database, and then translating these queries into natural language questions using a sequence-to-sequence model. However, a notable drawback of these methods is that the synthesized natural language questions often starkly differ from actual users. To bridge this gap, we propose a novel bi-directional data augmentation strategy. This approach not only leverages SQL-to-question synthesis but also incorporates question-to-SQL synthesis, more accurately generating the variety of questions real-world users might ask.

3 PRELIMINARIES

Text-to-SQL Task. The objective of text-to-SQL is to generate a SQL query S based on a natural language question Q and a database D, such that the SQL query can be executed to answer the question:

$$S = Parser(Q, D), \tag{1}$$

where the $Parser()$ is designed to interpret the provided Q using D and produce S. D contains database schema (*i.e.*, tables and columns) and database metadata, which contains column types, comments, column values, primary keys, and foreign key relations. An illustrative text-to-SQL example is presented in Figure 2.

Pre-trained Language Models. Language models, primarily based on the Transformer architecture [66], excel at text understanding and generation tasks. They typically undergo an initial pre-training phase on extensive text data using unsupervised learning objectives. Two prominent unsupervised learning paradigms are "language modeling" and "masked language modeling". In the former, models like GPT [52], PaLM [10], and LLaMa [64] predict the next word from a given context. In masked language modeling, specific words or spans within the input are randomly masked, and the task is to reconstruct the masked segments based on the unmasked context. Representative pre-trained language models using this approach include BERT [76], RoBERTa [43], and T5 [53].

Supervised Fine-Tuning and In-context Learning. Pre-trained language models possess extensive linguistic knowledge, yet specific tasks often demand unique language patterns or domain expertise. To address this, supervised fine-tuning (SFT) involves further training the model on task-specific labeled data, leveraging its initial pre-training and insights gained from the new dataset. In contrast to SFT, the concept of "in-context learning" [42, 70] enables language models to perform new tasks by simply providing appropriate prompts in the input, without the need for SFT. However, the effectiveness of in-context learning depends heavily on the quality of the prompts and the language model itself.

4 OVERVIEW

As illustrated in Figure 3, we introduce three components to solve the challenges in developing a compact but powerful text-to-SQL model and show the flexible usage of CodeS.

Incremental Pre-training (Figure 3(a) and Section 5). To improve existing language models' SQL generation and natural language understanding capabilities, we first collect a new corpus consisting of 11GB of SQL-related data, 6GB of NL-to-code data, and 4.5 GB of NL-related data from diverse sources. Then, based on StarCoder, we perform incremental pre-training using the SQL-centric corpus and obtain our pre-trained language model CodeS, which is available in 4 scales: 1B, 3B, 7B, and 15B.

Database Prompt Construction (Figure 3(b) and Section 6). We present a comprehensive database prompt construction approach to generate high-quality database prompts. The strategy mainly contains a schema filter and a value retriever. The schema filter is to eliminate irrelevant tables and columns based on the given question. The value retriever is tailored to extract potentially useful database values that align with the question. In addition to table and column names, we also incorporate various metadata, including data types, comments, representative column values, and information on primary and foreign keys. This comprehensive inclusion serves to provide a richer context for text-to-SQL models.

New Domain Adaptation (Figure 3(c) and Section 7). We present a bi-directional data augmentation method to produce a vast set of (question, SQL) pairs for new domain databases. In the question-to-SQL direction, we start with a few real-world questions, label

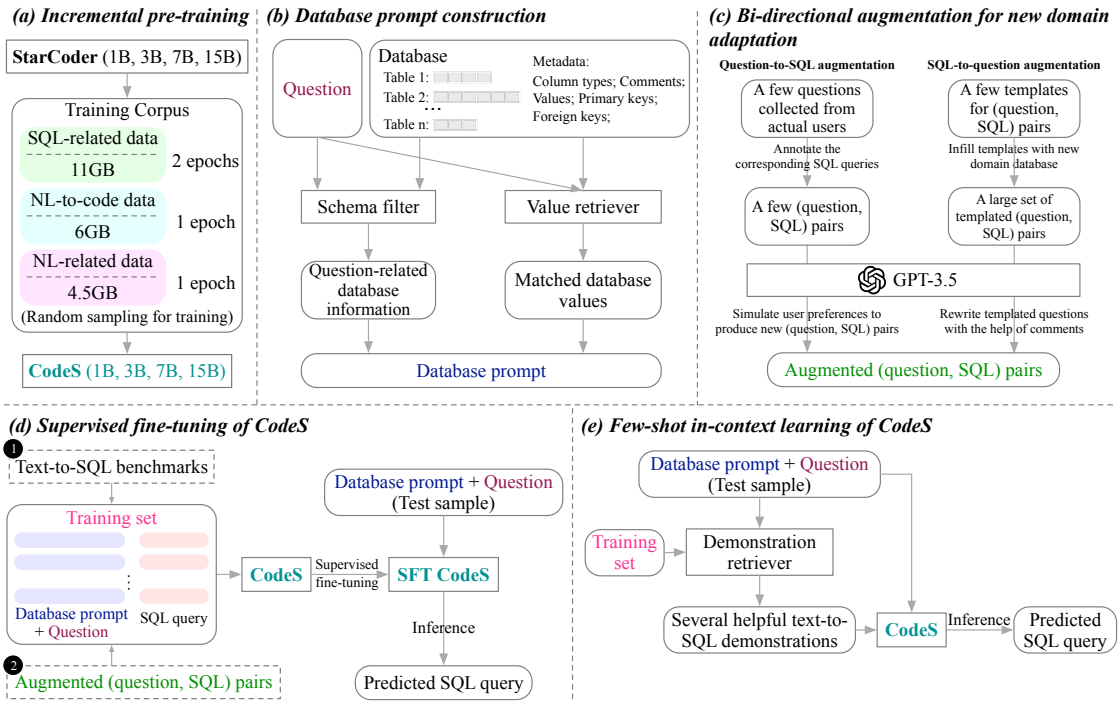


Figure 3: Illustration of the comprehensive framework which encompasses: (a) CodeS that is incrementally pre-trained on top of StarCoder using our specially curated SQL-focused dataset. (b) Our unique method for database prompt construction. (c) The proposed bi-directional data augmentation technique for adapting to new domains. CodeS can be employed in two distinct manners: (d) Inferring after a supervised fine-tuning of CodeS on a training dataset, sourced from text-to-SQL benchmarks along with our enriched (question, SQL) pairs. (e) Direct inference through few-shot in-context learning on CodeS.

their SQL counterparts, and use GPT-3.5 to expand the dataset. For the SQL-to-question approach, we extract templates from existing text-to-SQL benchmarks, infill the templates with the schema of the new database, and refine the questions with GPT-3.5. This bidirectional strategy creates a robust training set with minimal annotation, easing model fine-tuning for the new domain.

The Usage of CodeS (Figure 3(d), (e) and Section 8). In cases where abundant training data is accessible, we can directly fine-tune the parameters of the CodeS, tailoring it to closely match the distribution of labeled data (Figure 3(d)). Conversely, in scenarios with limited training data, we can utilize the CodeS's in-context learning ability without any fine-tuning by only offering a few text-to-SQL demonstrations (Figure 3(e)). A demonstration retriever is introduced to extract valuable demonstrations by concurrently considering both question similarity and question-pattern similarity.

Complexity Discussion. In discussing the complexities of our proposed solution, which is composed of various components, it's essential to distinguish between offline and online processes. Specifically, the incremental pre-training (Section 5) and new domain adaptation (Section 7) are conducted offline, meaning they are executed only once. Conversely, the database prompt construction strategy (Section 6) is an online process, as it must respond to each user's query during inference. The complexity of prompt construction arises from two main components: the schema filter and the value retriever. The schema filter employs a compact neural network for schema classification, which achieves fast inference speeds. On the other hand, the value retriever's efficiency is significantly enhanced by integrating BM25 indexing, leading to a noticeable acceleration in its online processing speed.

5 INCREMENTAL PRE-TRAINING

5.1 Pre-training Corpus

We enhance the text-to-SQL model's capabilities in SQL generation and natural language understanding through the collection of datasets from three key dimensions: SQL-related data, natural language-related data, and natural language-to-code data.

SQL-related data [11GB]. To further enhance the SQL generation capability of language models, we employ the SQL segment from StarCoder's pre-training corpus [39].

NL-related data [4.5GB]. To bolster the capability in natural language comprehension, we sample 4.5GB of high-quality dialog data from three sources: (1) Alpaca-cleaned[3] is designed for developing an instruction-following language model. This dataset is constructed using the self-instruct technique [69], aided by OpenAI's text-davinci-003 model. (2) Unnatural-instructions [28] is also a large instruction-following dataset collected with almost no human labor. Both alpaca-cleaned and unnatural-instructions datasets can

[3]https://huggingface.co/datasets/yahma/alpaca-cleaned

Table 1: Details about architectures of CodeS. "Shared": remains consistent across all model versions; "Distinct": varies among different model versions.

Type	Hyper-parameter	Value
Shared	Transformer architecture	Decoder-only
	Position embedding	Learned absolute embeddings
	Attention type	Multi-query
	FlashAttention-2	Enable
	Vocabulary size	49,152
Distinct	#Parameters	1B/3B/7B/15B
	Maximum context length	8,192/8,192/8,192/6,144
	Transformer's hidden size	2,048/2,816/4,096/6,144
	Feed-forward hidden size	8,192/11,264/16,384/24,576
	#Attention heads	16/22/32/48
	#Transformer blocks	24/36/42/40

be characterized as single-turn dialogues. (3) UltraChat [15] is a multi-turn dialogue dataset, produced by iteratively invoking two distinct GPT-3.5 APIs.

NL-to-code data [6GB]. To bridge the gap between natural language questions and SQL queries, we incorporate four types of NL-to-code datasets into our pre-training corpus: (1) CoNaLa [74] and StaQC [73] are derived automatically from Stack Overflow, encompasses many NL-to-Python and NL-to-SQL pairs. (2) CodeAlpaca-20k[4] encompasses a wealth of instruction-following data related to code, being created using the self-instruct methodology [69]. (3) Jupyter-structured-clean-dedup, a subset of the StarCoder's pre-training corpus, comprises a vast collection of structured Jupyter notebooks containing both code and accompanying natural language explanations. (4) Unlike the previously mentioned datasets, NL-SQL-458K is a brand-new dataset crafted by us, containing a vast number of NL-SQL pairs. Specifically, we start by using regular expressions to extract all "SELECT" queries from three extensive open-source corpora: The Pile [22], The Stack [31], and GitHub Code[5]. We then filter out queries with syntax errors, resulting in 458K SQL queries. To generate corresponding natural language questions for each SQL query, we employ GPT-3.5, using the prompts of eight paired (SQL, question) demonstrations.

5.2 Pre-Training Details

CodeS is built upon StarCoder, a series of open-source code language models pre-trained on a mixture of over 80 programming languages (such as C, Python, Java, PHP, SQL, and others), Jupyter notebooks, GitHub issues, and Git commits. To develop CodeS, we incrementally pre-train StarCoder for two epochs on SQL-related data and one epoch each on NL-related and NL-to-code data. This mixed training in natural language and code offers benefits for a wide range of tasks in both domains [45]. Specifically, CodeS-15B is based on StarCoder-15B, while CodeS-(1B, 3B, 7B) are derived from the respective StarCoderBase-(1B, 3B, 7B). Then we optimize the language modeling objective that is widely used in prior pre-trained language models like GPT [52] and LLaMA [64]. Specifically, given a sequence x consisting of n tokens, denoted as $t_0, t_1, t_2, ..., t_{n-1}$, our objective is to maximize the likelihood of the entire sequence. This is achieved by calculating the product of the conditional probabilities for each token:

$$p(x) = \prod_{i=1}^{n-1} p(t_i | t_1, t_2, ...t_{i-1}). \quad (2)$$

To optimize the objective, we employ the AdamW optimizer [44] with parameters set to $\beta_1 = 0.9$, $\beta_2 = 0.95$, and $\epsilon = 10^{-8}$. The learning rate is configured at $5e^{-5}$, accompanied by a weight decay of 0.1. Our learning rate scheduler follows a cosine decay without any warm-up steps, and we set the final learning rate at a tenth of its initial value. The training process uses a large batch size comprising 4M tokens, and to ensure stability, we apply gradient clipping with a clipping value of 1.0. We leverage the DeepSpeed Zero-3 framework [54], employing BF16 mixed precision, to optimize GPU memory consumption during pre-training. More details about model architectures can be found in Table 1. We integrate the FlashAttention-2 [11] into CodeS, enhancing its capability to handle extended context lengths. However, due to GPU memory constraints, CodeS-(1B, 3B, 7B) can handle a maximum context length of 8,192, whereas CodeS-15B is limited to 6,144. In practice, incremental pre-training over our collected corpus takes approximately 1.5, 3, 8, 16 days for CodeS-(1B, 3B,7B, 15B).

6 DATABASE PROMPT CONSTRUCTION

Beyond model advancements, building effective database prompts is crucial for the text-to-SQL task. High-quality prompts furnish the language model with valuable insights, enabling it to generate precise SQL queries more efficiently. To craft these superior database prompts, we employ two key strategies: a schema filter and a value retriever, while also incorporating crucial database metadata. The pseudo-code detailing our prompt construction process is presented in Algorithm 1.

6.1 Schema Filter

In real-world scenarios, databases often encompass a vast array of tables and columns, resulting in extremely long database prompts. When these prompts surpass the language model's maximum context length, truncation becomes necessary. However, this process may omit vital tables or columns that are necessary for generating the target SQL query. Hence, it's imperative to adopt a method that minimizes the database prompt length without sacrificing critical schema information. Following [36], we employ a schema filter to retain the most relevant tables and columns within the database for each text-to-SQL sample. Concretely, given a database and a question, we develop a schema item classifier, which is trained to predict relevance scores for the tables and columns according to the user's question. Utilizing these scores, we retain the top top_{k1} tables and, for each of these tables, the top top_{k2} columns. Then, for text-to-SQL samples in the training set, since the ground truth SQL query is available, we could directly identify the used tables and columns. Aiming for consistency in distribution between test and training data, if the number of the used tables falls below top_{k1}, we incorporate randomly selected, unused tables from the database as padding. A similar procedure is adopted for columns, ensuring each retained table contains top_{k2} columns. The integration of the

[4]https://huggingface.co/datasets/sahil2801/CodeAlpaca-20k
[5]https://huggingface.co/datasets/codeparrot/github-code

schema filter not only reduces the length of the database prompts but also alleviates the schema-linking pressure for the model.

6.2 Value Retriever

Retrieving values from the database that align with the question can help the language model perform better schema linking when generating predicates. For a natural language question in the BIRD benchmark "`How many clients opened their accounts in Jesenik branch were women?`", a comparison against the database's values reveals that the "`a2`" column in the "`district`" table holds the value "`Jesenik`". Subsequently, integrating the information "`district.a2 = 'Jesenik'`" into the database prompt can guide the model in producing accurate predicates for the SQL query. Prior work [36, 37, 57] consistently uses the longest common substring (LCS) algorithm to calculate the matching degree of a database value to a question. However, the time complexity of this algorithm is $O(f * u)$, where f and u denote the lengths of the two input strings. In many scenarios where the database contains a vast amount of values (for instance, the Donor database in the BIRD benchmark, which encompasses approximately 116.5 million valid values), using LCS to calculate the matching degree for every database value becomes excessively time-consuming. To tackle this, we propose a "coarse-to-fine" matching approach. The essence of this method lies in utilizing indexes for a fast yet coarse-grained initial search, followed by a fine-grained matching process using the LCS algorithm. Specifically, we use Lucene[6] to build the BM25 index for all values stored in each database. When a user's question is received, the indexes first extract hundreds of potentially relevant values from the whole database based on the question. Then, the LCS method is employed to calculate the degree of match between the question and these potentially relevant values to find the most relevant values. The integration of the BM25 index significantly enhances the value retrieval speed for the extensive database by drastically reducing the number of LCS algorithm invocations from potentially millions to just hundreds.

6.3 Database Metadata

In our database prompt, we additionally incorporate certain metadata that is valuable for text-to-SQL:

(1) Column Data Types: The data type of a column dictates its validation rules and permissible operations. For instance, numeric types like `INTEGER` and `REAL` support arithmetic operations, whereas string types don't. If certain data is stored as a string type, the `CAST` function must be used in the SQL query before performing arithmetic operations on it.

(2) Comments: The ambiguities in database schemas are prevalent in real-world databases. Table 2 shows examples of ambiguous columns from the BIRD benchmark. Ambiguous schemas can lead models to select the wrong tables or columns in their generated SQL queries, as language models typically use semantic similarity for schema linking. Fortunately, databases usually provide informative comments for ambiguous schema. We incorporate these comments into both the input of the schema item classifier and the database prompts to facilitate the LLM to perform accurate schema linking.

[6]https://github.com/castorini/pyserini

Table 2: Examples of ambiguous columns in BIRD dataset.

Database	Column name	Comment
language_corpus	w1st	word id of the first word
hockey	rotl	road overtime loses
ice_hockey_draft	pim	penalty minutes

(3) Representative Database Values: In addition to column names, representative column values are beneficial. For instance, to generate predicates such as "`client.gender = 'F'`", it's imperative to inform the language model that the "`gender`" column offers two values: 'M' and 'F'. Similarly, for predicates like "`SUBSTR (hiredate, 1, 4) = '2009'`", the model should be aware of the "hiredate" column's specific format, "YYYY-MM-DD". Apparently, question-matched values don't always address these nuances. To address this, we extract representative values for each column. Drawing inspiration from [6], we employ the SQL query "`SELECT DISTINCT {COLUMN} FROM {TABLE} WHERE {COLUMN} IS NOT NULL LIMIT 2`" to capture two distinct representative non-null values from each column. By offering these insightful values, the language model is better positioned to produce precise and context-relevant SQL queries.

(4) Primary and Foreign Keys: The primary key uniquely identifies each row in a table, while the foreign key creates associations between tables. Incorporating primary and foreign key information can guide the model to deduce the appropriate join path, ensuring accurate `JOIN ON` clause generation. In practice, we use a unique identifier for every primary key column and represent each foreign key as "`{TABLE1}.{COLUMN1} = {TABLE2}.{COLUMN2}`" within the database prompt.

Algorithm 1: Database Prompt Construction

Input: user question Q, schema item classifier model M_{cls}, database schema D_{schema}, database metadata D_{meta}, pre-built BM25 index I, maximum table and column number top_{k1}, top_{k2}

Output: Database prompt $Prompt_{db}$

// Apply schema filter

$computeRelevantScores(Q, D_{schema}, M_{cls}) \rightarrow Scores$;

$filterDBInfo(Scores, D_{schema}, D_{meta}, top_{k1}, top_{k2}) \rightarrow D^r_{schema}, D^r_{meta}$;

// Apply value retriever

$BM25Matching(Q, I) \rightarrow V_{coarse-grained}$;

$LCS(Q, V_{coarse-grained}) \rightarrow V_{fine-grained}$;

// Serialization and concatenation

$Serialize(D^r_{schema}, D^r_{meta}) \rightarrow S_{db}$;

$Serialize(V_{fine-grained}) \rightarrow S_{value}$;

$ConcatSequence(S_{db}, S_{value}) \rightarrow Prompt_{db}$;

return $Prompt_{db}$

In Figure 4, we show a training sample from the Spider benchmark, consisting of paired input and output sequences. This database prompt is crafted using our proposed strategy. As seen, based on the user's question, a relevant database value, `Sarah Martinez`, is extracted from the `reviewer.name` column. Then, the displayed primary and foreign keys further guide the language model in generating the `JOIN ON` clauses.

INPUT

Database prompt:

table **movie** , columns = [movie.mid (int | primary key | comment : movie id | values : 101 , 102) , movie.title (text | values : Gone with the Wind , Star Wars) , movie.year (int | values : 1939 , 1977) , movie.director (text | values : Victor Fleming , George Lucas)]
table **reviewer** , columns = [reviewer.rid (int | primary key | comment : reviewer id | values : 201 , 202) , reviewer.name (text | values : Sarah Martinez , Daniel Lewis)]
table **rating** , columns = [rating.rid (int | comment : reviewer id | values : 201 , 202) , rating.mid (int | comment : movie id | values : 101 ,106) , rating.stars (int | comment : rating stars | values : 2 , 4) , rating.ratingdate (date | values : 2011-01-22 , 2011-01-27)]
foreign keys :
rating.rid = reviewer.rid
rating.mid = movie.mid
matched values :
reviewer.name (Sarah Martinez)

Question:

What are the names of all directors whose movies have been reviewed by Sarah Martinez?

OUTPUT

```
SELECT DISTINCT movie.director FROM rating JOIN movie ON rating.mid = movie.mid
JOIN reviewer ON rating.rid = reviewer.rid WHERE reviewer.name = 'Sarah Martinez'
```

Figure 4: A text-to-SQL sample in Spider's training set, consisting of a triplet of <database prompt, question, SQL query>. The database prompt is crafted by our proposed method.

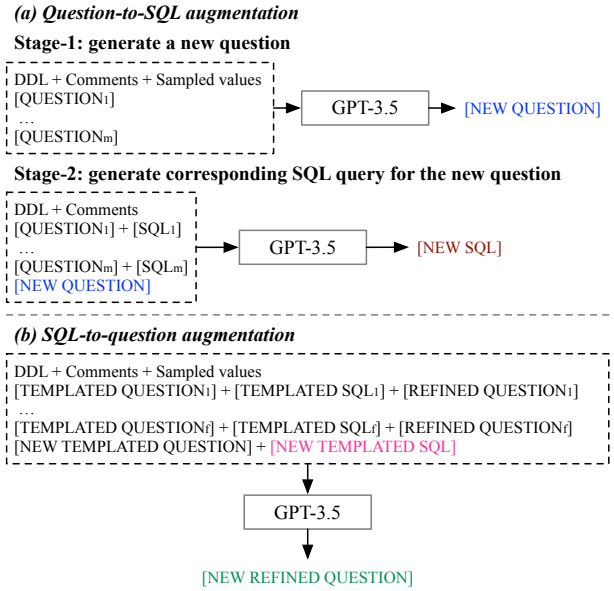


Figure 5: Prompt formats used in the bi-directional data augmentation. DDL stands for data definition language.

7 NEW DOMAIN ADAPTION

In real-world scenarios, people usually have their own databases from various new domains such as finance, genes, biology, academia, and more. However, developing a text-to-SQL model on these databases is challenging because of the lack of labeled training data. In this section, we propose a bi-directional data augmentation technique to automatically generate a large set of authentic and general (question, SQL) pairs with minimal annotation costs.

Question-to-SQL Augmentation. This augmentation direction seeks to produce genuine (question, SQL) pairs aligned with user preferences. Specifically, we first gather a few authentic and representative natural language questions from real-world text-to-SQL users. These questions embody the most common inquiries users have for specific databases. Then, we manually annotate their corresponding SQL queries to obtain a few high-quality (question, SQL) pairs. Given that frequently asked questions are typically few in number, the annotation effort is relatively minimal. Furthermore, given the constrained volume of annotated data, it's insufficient for directly fine-tuning language models. To address this, we introduce a two-stage prompting approach: Initially, we prompt GPT-3.5 to generate potential questions, drawing inspiration from the real questions we've gathered, effectively capturing user intents. Then, we employ GPT-3.5 to provide corresponding SQL queries for these synthesized questions. Figure 5(a) shows the prompts used during the two-stage process. Here, [QUESTIONi] and [SQLi] denote a pair of manually labeled data, with m indicating the total number of such pairs. To guarantee the diversity of the newly generated questions, we shuffle the sequence of user-providing questions and employ a high-temperature hyperparameter for each generation. Lastly, [NEW QUESTION] and [NEW SQL] represent a new data pair that mirrors user preferences, ensuring its authenticity.

SQL-to-Question Augmentation. Inspired by SyntaxSQLNet [78], we explore another augmentation method that generates generic (question, SQL) pairs using a set of universal templates. Specifically, this paper uses (question, SQL) templates derived from Spider, a widely-recognized text-to-SQL benchmark, encompassing 75 common SQL templates. Examples include a templated question: "`Return the lowest {COLUMN} of {TABLE}`", and its corresponding templated SQL: "`SELECT {COLUMN} FROM {TABLE} GROUP BY {COLUMN} ORDER BY COUNT (*) ASC LIMIT 1`". Given the versatility of natural language, a single SQL template often aligns with multiple question templates. Next, we fill slots with the new database schema to generate numerous template (question, SQL) pairs. However, these templated questions can seem artificial since they directly insert table and column names. To make these questions more natural, we leverage GPT-3.5 to rephrase them based on f manually crafted refinement examples. As showned in Figure 5(b), each example comprises a triplet: [TEMPLATED QUESTION$_i$], [TEMPLATED SQL$_i$], and [REFINED QUESTION$_i$]. The end result is a new pair, [NEW REFINED QUESTION] and [NEW TEMPLATED SQL], which align more closely with typical text-to-SQL datasets.

8 USAGE OF CODES

We use CodeS by fine-tuning and few-shot in-context learning. Formally, given training set $D = \{d_1, d_2, ..., d_h\}$ with h text-to-SQL samples, each sample is a triplet $(d_i^{db}, d_i^q, d_i^{sql})$ denoting the database, question, and associated SQL query. For a test sample t, consisting of database t^{db} and question t^q, we aim to generate SQL query t^{sql}. Before employing CodeS, we first convert each database in the text-to-SQL sample into its corresponding database prompt (see Section 6). Thus, each training sample is represented as a triplet: $(d_i^{dbp}, d_i^q, d_i^{sql})$, with d_i^{dbp} being the prompt for database d_i^{db}. Similarly, each test sample is transformed into the pair (t^{dbp}, t^q), where t^{dbp} serves as the prompt for database t^{db}.

8.1 Supervised Fine-Tuning

Given a substantial collection of training data, supervised fine-tuning (SFT) is a preferred option as it allows rapid adaptation to

the specific data distribution. First, we form the input sequence as the combination of the database prompt and the question. Then, CodeS is optimized to predict the desired SQL query based on this input sequence. Therefore, the learning objective of SFT CodeS is:

$$Loss = \frac{1}{h}\sum_{i=1}^{h} p(d_i^{sql} | d_i^{dbp}, d_i^q). \quad (3)$$

After the fine-tuning process, for the given test sample, the refined CodeS can readily produce the SQL query using the combined inputs of t^{dbp} and t^q.

8.2 Few-shot In-Context Learning

In cases where fine-tuning is impractical, we can utilize the model's built-in text-to-SQL capabilities without any fine-tuning. The efficacy of few-shot learning isn't solely based on the model's intrinsic capabilities; it's also influenced by the examples used [41, 70, 84]. Thus we employ an efficient retriever to source valuable demonstrations.

Our goal is to select K useful examples from the dataset D to aid the model in predicting the right SQL query. A basic way is to check the semantic relevance between the test question, t^q, and all training questions, $\{d_1^q, d_2^q, ..., d_h^q\}$, evaluating the top-K training samples that match best. However, this can overly prioritize entities, leading to demonstrations that simply reflect the test question's entities. For instance, a question asking for singers born in 1948 or 1949 might fetch a training question about an artist who sang the most songs, due to shared references to "`singers and songs`".

To avoid overemphasizing entities, we focus on the question's core structure by stripping its entities. For example, we aim to enable the most suitable demonstration question for the question about singers born in 1948 or 1949 as "`Show the names of members from either 'United States' or 'Canada'`" without being limited to "`singers and songs`".

Formally, we define the similarity score between the test question t^q and the training question d_i^q as:

$$max(sentsim(t^q, d_i^q), sentsim(t^{qs}, d_i^{qs})), \quad (4)$$

where t^{qs} and d_i^{qs} represent the extracted question patterns from t^q and d_i^q, respectively. Using nltk's tool, we identify and remove entities from questions to get their patterns. We then use SimCSE [23], a sentence similarity model, to compute sequence similarities. We term this enhanced retrieval approach the "question-pattern-aware demonstration retriever".

Finally, after selecting the K most relevant examples, we merge them with the test sample, creating a unified input sequence, which is then fed into the pre-trained model to derive the SQL query.

9 EXPERIMENTS

9.1 Experimental Setup

9.1.1 Datasets. We perform main experiments on two English text-to-SQL benchmarks: Spider [79] and BIRD [38]. We also assess our models' robustness across four more challenging benchmarks: Spider-DK [20], Spider-Syn [19], Spider-Realistic [12], and Dr.Spider [7]. Following experimental settings in previous studies [25, 36], we utilize Spider as the training set and evaluate our models against these robustness-diagnostic test sets. Moreover, we manually created two databases from financial and academic domains, named Bank-Financials and Aminer-Simplified.

Spider offers a training set comprising 8,659 samples, a development set with 1,034 samples, and a hidden test set. The training portion encompasses 7,000 manually annotated samples, supplemented by an additional 1,659 samples sourced from six previous text-to-SQL datasets: Restaurants [50, 63], GeoQuery [80], Scholar [30], Academic [35], IMDB, and Yelp [72]. Spider contains 200 databases that cover 138 diverse domains. However, due to the hardware constraints of the Spider submission platform[7], we can not evaluate our models against its test set. Consequently, for Spider, primary evaluations utilize its publicly available development set.

BIRD comprises a training set of 9,428 samples, a development set with 1,534 samples, and a hidden test set. BIRD encompasses 95 databases, cumulatively accounting for 33.4GB across 37 distinct professional domains. BIRD is more challenging, with each of BIRD's databases containing around 549K rows on average, in contrast to Spider's limited capacity of just 2,000 rows. Moreover, BIRD offers **external knowledge (EK)** for specific samples to facilitate the generation of the right SQL query. Since it's impractical for users to supply such external knowledge, our evaluations on BIRD are categorized into two scenarios: with and without external knowledge. When the external knowledge is used, we simply integrate it with the question, yielding an enriched input prompt. We provide our code and models to the official organizers of BIRD for evaluation on their hidden test set.

Spider-DK, Spider-Syn, Spider-Realistic are variants derived from the original Spider dataset. They are specifically designed to mimic questions that could be posed by users in real-world scenarios. **Dr.Spider**, also a Spider derivative, incorporates 17 distinct perturbations across questions, databases, and SQL queries to holistically evaluate the robustness of text-to-SQL models. Specifically, Dr.Spider comprises 3 test sets with database perturbation, 9 test sets reflecting question perturbation, and 5 test sets with SQL perturbation.

Bank-Financials derives from a finance database containing 4 tables, with the largest table containing 65 columns (see Figure 2). We generate a training set comprising 4,901 samples for the new finance database by using the data augmentation method proposed in Section 7. For evaluation, we further manually annotate 91 real-world questions as the test set. **Aminer-Simplified** originates from an academic database, sampled from a large-scale academic graph, AMiner [60, 62, 81, 82]. We follow the same procedure as Bank-Financials and obtain a training set with 5,427 samples and a test set with 97 samples. Both training sets of these two datasets are derived from only 30 manually annotated samples.

9.1.2 Evaluation Metrics. (1) For Spider-family benchmarks (including Spider, Spider-DK, Spider-Syn, Spider-Realistic, and Dr.Spider), we consider two prevalent evaluation metrics: execution accuracy (EX) [79] and test-suite accuracy (TS) [85]. The EX metric evaluates whether the predicted and ground-truth SQL queries yield

[7]The Spider benchmark utilizes CodaLab Worksheets as its submission platform, which can be accessed at https://worksheets.codalab.org/home. This platform offers GPUs equipped with 12GB of memory. However, our best model, SFT CodeS-15B, requires at least 35GB of GPU memory to perform inference.

the same execution results on the database. However, there can be instances where EX gives false positives — situations where an incorrect SQL prediction coincidentally produces the same output as the correct query [85]. To counteract this, the test-suite accuracy was developed. It assesses if the generated SQL query consistently passes the EX evaluations across multiple database instances, which are derived from automated database augmentations. Due to its proficiency in reducing false positives, TS stands out as a more trustworthy metric than EX. It's worth noting that for Spider-DK and Dr.Spider, the TS script lacks test suites for their respective databases. As a result, we exclusively adopt the EX metric for them. (2) The BIRD benchmark primarily relies on execution accuracy (EX) as its evaluation metric, because the databases in BIRD typically contain a large number of values, minimizing the chances of false positives. Additionally, BIRD introduces the valid efficiency score (VES) to assess the execution efficiency of accurately generated SQL queries. Unlike EX, in VES, the score for an accurately generated SQL query is no longer 1 but is determined by the execution time of the ground truth SQL query divided by the execution time of the predicted SQL query. Thus, if the execution efficiencies are the same, the scores for EX and VES will be identical. However, if the predicted SQL query executes faster than the ground truth SQL query, the VES value will exceed that of EX. In practice, each correct predicted SQL query and its ground truth counterpart are executed 100 times, with their run times recorded. Yet, preliminary experiments indicated that VES could be highly susceptible to fluctuations based on varying hardware, software, and system status. Hence, for BIRD, EX serves as the stable and dependable metric.

9.1.3 Evaluation Settings. We evaluate CodeS under both few-shot, in-context learning and supervised fine-tuning scenarios. The few-shot in-context learning provides insight into the inherent SQL generation capabilities of language models, as it doesn't involve any fine-tuning. Instead, the models rely on their pre-training knowledge to address the users' questions. Then, to assess the alignment ability of CodeS, we fully fine-tune it using the training set and evaluate the fine-tuned version on different dev and test sets.

9.1.4 Implementation details. In our experiments, we utilize SQLite to host and manage all databases. The training and evaluation processes for the schema item classifier are conducted in accordance with [36]. Specifically, for each dataset, we train a classifier on its respective training set and assess its classification accuracy on the development set, utilizing the AUC metric for evaluation. The results of this evaluation are presented in Table 3. It is observed that the AUC scores for Spider consistently surpass those for BIRD (and BIRD w/ EK). We hypothesize that this disparity is stemmed from the prevalence of ambiguous schemas in BIRD. Additionally, incorporating external knowledge enhances classification accuracy in BIRD, especially when the external knowledge directly highlights tables and columns relevant to the question. In supervised CodeS fine-tuning, we set top_{k1} and top_{k2} to 6 and 10. For few-shot in-context learning, to accommodate more demonstrations in the input prompt, these are adjusted to 5 and 6. We then fine-tune CodeS for 4 epochs with a batch size of 128, a learning rate of $5e^{-6}$, and a max context length of 4,096. SFT model performance might be enhanced with hyperparameter tuning. The learning rate has a cosine decay and a linear warmup for the initial 5% of training.

Table 3: Table and column AUC scores for the trained schema item classifiers.

	Spider	BIRD	BIRD w/ EK
Table AUC	**0.991**	0.966	0.976
Column AUC	**0.993**	0.943	0.957

Other optimization settings align with Section 5.2. For generation, a beam search produces 4 SQL candidates, picking the first executable one as the outcome.

9.1.5 Enviroments. Our experiments are conducted using PyTorch 1.13.1 on a computer running the CentOS Linux 7 operating system, equipped with 8 NVIDIA A800 80GB GPUs, an Intel(R) Xeon(R) Platinum 8358 CPU, and 1024GB of RAM.

9.1.6 Baselines. For few-shot in-context learning, our objective is to gauge the inherent SQL generation capabilities of CodeS. Therefore, our baselines consist of widely recognized open-source language models, such as those from the StarCoder [39], CodeGen [45, 46], and Llama2 [65]. For supervised fine-tuning, almost all baselines are drawn from the SOTA text-to-SQL approaches listed on the official leaderboards of benchmarks. We additionally fine-tune Llama2-7B and 13B as our competitive baselines.

9.2 Evaluation on Few-shot In-Context Learning

Table 4 shows the results of few-shot in-context learning evaluations on the Spider and BIRD benchmarks. We increase the demonstrations from 1 to 3 to 5. For a fair comparison, all models use our few-shot in-context learning framework with consistent input prompts. When comparing various versions of StarCoder (*i.e.*, before incremental pre-training) with our CodeS (*i.e.*, after incremental pre-training), it's clear that incremental pre-training greatly improves StarCoder's SQL generation capability. *Consequently, CodeS-15B emerges as the leading open-source pre-trained language model in SQL generation.* Furthermore, smaller models exhibit a more pronounced improvement compared to larger models. This observation underscores our initial hypothesis that smaller models, due to their constrained parameters, may not be optimally trained for SQL-related tasks.

9.3 Evaluation on Supervised Fine-Tuning

Table 5 presents the evaluation results in EX and TS on Spider's dev set. Remarkably, SFT CodeS-3B outperforms the leading GPT-4-based method (*i.e.*, DIN-SQL and DAIL-SQL), illustrating the potential of smaller models to excel as text-to-SQL parsers after fine-tuning. Furthermore, SFT CodeS-7B and SFT CodeS-15B achieve new SOTA performance on Spider's development set. However, SFT CodeS-7B exhibits a marginal advantage over SFT CodeS-15B, suggesting potential overfitting of CodeS-15B to the Spider training data, which might slightly compromise its generalization to the development set. This phenomenon indicates that a larger model doesn't always guarantee better fine-tuning results.

Table 6 presents the evaluation results in EX and VES on BIRD's development and hidden test sets. Given BIRD's higher complexity compared to Spider, our approach yields more significant improvements over the baseline methods. Without using external

Table 4: In-context learning performance on Spider's and BIRD's dev sets using 1-shot, 3-shot, and 5-shot settings. Due to space constraints, we only present the TS (%) results for Spider and the EX (%) results for BIRD and BIRD w/ EK (with external knowledge). The top performance is emphasized in bold, while the runner-up is underlined.

LLMs	Spider's dev (TS%)			BIRD's dev (EX%)			BIRD's dev w/ EK (EX%)		
	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot
StarCoderBase-1B [39]	43.7	46.8	48.6	17.21	18.84	20.08	15.78	21.90	22.69
StarCoderBase-3B [39]	58.5	60.0	60.8	23.01	26.01	26.27	27.18	32.72	36.31
CodeGen-mono-6B [46]	44.6	46.7	45.0	14.47	16.69	15.91	13.10	15.78	16.04
StarCoderBase-7B [39]	59.7	63.1	64.6	26.66	29.79	30.44	32.99	39.24	40.61
CodeGen2-7B [45]	46.8	49.8	50.8	18.38	18.90	19.30	16.56	19.88	19.56
Llama2-7B [65]	34.9	39.3	40.2	12.45	16.30	15.12	15.25	19.04	19.88
Llama2-13B [65]	45.4	48.5	47.6	16.88	20.34	20.47	21.06	25.81	25.36
StarCoderBase-15B [39]	63.5	67.7	70.0	29.86	31.55	33.77	35.40	39.24	41.20
StarCoder-15B [39]	63.8	67.6	69.6	29.86	32.99	33.64	35.46	40.09	42.44
StarCoderPlus-15B [39]	58.3	63.3	65.5	27.51	30.57	31.68	32.86	39.31	41.53
CodeGen-mono-16B [46]	50.9	52.8	52.4	20.34	21.58	22.23	22.95	25.42	23.92
CodeGen2-16B [45]	51.6	55.9	57.4	21.45	22.69	23.14	23.01	25.42	25.23
CodeS-1B	55.7 (↑12.0)	57.4 (↑10.6)	59.1 (↑10.5)	22.23 (↑5.02)	25.42 (↑6.58)	27.18 (↑7.1)	25.62 (↑9.84)	29.47 (↑7.57)	31.03 (↑8.34)
CodeS-3B	64.6 (↑6.1)	67.8 (↑7.8)	69.7 (↑8.9)	29.07 (↑6.06)	31.23 (↑5.22)	31.81 (↑5.54)	35.59 (↑8.41)	39.18 (↑6.46)	41.85 (↑5.54)
CodeS-7B	66.0 (↑6.3)	69.8 (↑6.7)	71.8 (↑7.2)	30.77 (↑4.11)	33.44 (↑3.65)	34.49 (↑4.05)	37.29 (↑4.30)	41.98 (↑2.74)	44.26 (↑3.65)
CodeS-15B	**69.3** (↑5.5)	**71.5** (↑3.9)	**73.4** (↑3.8)	**34.09** (↑4.23)	**35.14** (↑2.15)	**37.03** (↑3.39)	**39.57** (↑4.11)	**43.48** (↑3.39)	**45.44** (↑3.00)

Table 5: Evaluation of SFT CodeS on Spider's dev set.

Methods	EX%	TS%
Fine-tuning-based methods		
T5-3B + PICARD [57]	79.3	69.4
RESDSQL-3B + NatSQL [36]	84.1	73.5
Graphix-T5-3B + PICARD [37]	81.0	75.0
Fine-tuned SQL-PaLM [61]	82.8	78.2
SFT Llama2-7B [65]	77.8	73.0
SFT Llama2-13B [65]	81.6	76.6
Prompting-based methods		
GPT-4 (few-shot) [51]	76.8	67.4
C3 + ChatGPT [16]	81.8	71.4
Self-Debug + Codex davinci [9]	84.1	-
DIN-SQL + GPT-4 [51]	82.8	74.2
DAIL-SQL + GPT-4 [21]	83.1	76.6
SQL-PaLM (few-shot) [61]	82.7	77.3
DAIL-SQL + GPT-4 + Self-Consistency [21]	83.6	76.2
Ours		
SFT CodeS-1B	77.9	72.2
SFT CodeS-3B	83.4	78.1
SFT CodeS-7B	**85.4**	**80.3**
SFT CodeS-15B	84.9	79.4

Table 6: Evaluation of SFT CodeS on BIRD's dev/test sets.

	Dev		Dev w/ EK		Test		Test w/ EK	
Methods	EX%	VES%	EX%	VES%	EX%	VES%	EX%	VES%
Baseline methods								
Fine-tuned T5-3B [38]	10.37	13.62	23.34	25.57	11.17	15.17	24.05	27.80
PaLM-2 [38]	-	-	27.38	-	-	-	33.04	-
Codex 175B [38]	25.42	33.37	34.35	43.41	24.86	35.40	36.47	41.60
ChatGPT [38]	24.05	27.97	42.24	-	26.77	36.68	39.30	51.40
ChatGPT + CoT [38]	25.88	32.33	36.64	42.30	28.95	49.69	40.08	56.56
Claude-2 [38]	-	-	42.70	-	-	-	49.02	-
GPT-4 [38]	-	-	49.15	-	-	-	54.89	60.77
DIN-SQL + GPT-4 [51]	-	-	50.72	58.79	-	-	55.90	59.44
DAIL-SQL + GPT-4 [21]	-	-	54.76	56.08	-	-	57.41	61.95
SFT Llama2-7B [65]	35.53	36.09	45.37	46.98	-	-	-	-
SFT Llama2-13B [65]	41.85	44.00	53.91	58.77	-	-	-	-
Ours								
SFT CodeS-1B	38.46	41.77	50.46	51.07	-	-	-	-
SFT CodeS-3B	43.42	44.55	55.02	56.54	-	-	-	-
SFT CodeS-7B	45.24	48.13	57.17	58.80	50.25	54.84	59.25	63.62
SFT CodeS-15B	**47.91**	**49.60**	**58.47**	**59.87**	**52.15**	**56.99**	**60.37**	**64.22**

knowledge, SFT CodeS-15B significantly outperforms ChatGPT + COT, improving from 28.95% to 52.15% in EX on the test set, a notable increase of 23.20%. When incorporating external knowledge (w/ EK), both SFT CodeS-7B and SFT CodeS-15B show clear EX improvements of 3.35% and 4.47%, respectively, on the test set compared to the previous SOTA text-to-SQL method, DIN-SQL + GPT-4. However, even though SFT CodeS-15B outperforms SFT CodeS-7B, the margin between them remains minimal, especially considering the doubled parameter size in the former. This suggests that CodeS-7B offers an optimal trade-off between computational efficiency and text-to-SQL capabilities. Additionally, the VES metric surpassing EX signifies that our models are more capable than human experts in producing execution-efficient SQL queries. At the time of writing, SFT CodeS-15B and SFT CodeS-7B hold the top positions on BIRD's official leaderboard.

9.4 Evaluation on Robustness Benchmarks

Table 7 evaluates SFT CodeS on three Spider variants for robustness: Spider-DK, Spider-Syn, and Spider-Realistic. Notably, SFT CodeS-7B exhibits exceptional performance, achieving gains of 2.6% on Spider-Syn (67.4% to 70.0%), 4.0% on Spider-Realistic (73.2% to 77.2%), and 4.5% on Spider-DK (67.5% to 72.0%), comparing with the best baselines. Even the SFT CodeS-3B outperforms previous SOTA methods across all the datasets. Considering that SFT CodeS is trained on Spider but tested on its variants, these results highlight

Table 7: Evaluation of SFT CodeS on Spider variants.

	Spider-Syn		Spider-Realistic		Spider-DK
Methods	EX%	TS%	EX%	TS%	EX%
T5-3B + PICARD [57]	69.8	61.8	71.4	61.7	62.5
RESDSQL-3B + NatSQL [36]	76.9	66.8	81.9	70.1	66.0
ChatGPT [40]	58.6	48.5	63.4	49.2	62.6
SQL-PaLM (Few-shot) [61]	74.6	67.4	77.6	72.4	66.5
SQL-PaLM (Fine-tuned) [61]	70.9	66.4	77.4	73.2	67.5
SFT CodeS-1B	66.5	59.3	70.9	61.8	64.7
SFT CodeS-3B	75.7	69.0	79.9	74.4	71.8
SFT CodeS-7B	76.9	**70.0**	82.9	**77.2**	**72.0**
SFT CodeS-15B	**77.0**	69.4	**83.1**	75.6	70.7

the model's impressive generalization capability in challenging distribution shift scenarios.

For a more comprehensive evaluation of the robustness of SFT CodeS, we further test SFT CodeS on Dr.Spider. The results can be found in Table 8. First, for the database (DB) perturbation, CodeS slightly lags behind ChatGPT + ZeroNL2SQL, mainly due to the *DBcontent-equivalence* perturbation. ChatGPT + ZeroNL2SQL uses a dense retriever for better semantic accuracy, but it's resource-intensive, needing more disk space and computation time. To maintain efficiency and real-world applicability, we opt for a sparse retriever. In the natural language question perturbation, both SFT CodeS-7B and SFT CodeS-15B outperform the previous best, ChatGPT + ZeroNL2SQL, scoring 74.3% and 75.2% versus 73.2%. This suggests our models have a better grasp of question semantics, leading to more accurate SQL queries. For SQL perturbations, our models slightly lag behind RESDSQL-3B + NatSQL. The latter's intermediary SQL representation, NatSQL, is more simple than SQL and aids its performance. However, its syntax is limited to Spider, making it less adaptable. In global average performance, SFT CodeS-7B and SFT CodeS-15B slightly surpass the prior best, ChatGPT + ZeroNL2SQL, which is tailored for robust text-to-SQL. Overall, even without a specific robustness design, our models frequently excel against methods built especially for text-to-SQL resilience.

9.5 Ablation studies

We conduct an extensive ablation study on Spider and BIRD under the 3-shot in-context learning setting to evaluate the impact of each key component. The results are shown in Table 9.

Demonstration Retriever. Using only question similarity for retrieving demonstrations (-w/o pattern similarity) results in a performance drop on Spider but less so on BIRD. This could be because Spider has less question variety than BIRD, making it easier to group similar text-to-SQL samples based on their patterns. Then, when replacing our demonstration retrieval strategy with a purely random selection (-w/o demonstration retriever), performance decreases in most scenarios.

Schema Filter and Value Retriever. We explore the effects of the schema filter and value retriever on performance. Without the schema filter, there's both a drop in performance and a slower generation speed due to longer input sequences. Omitting the value retriever also leads to a marked decrease in text-to-SQL performance, especially on the BIRD benchmark, highlighting its crucial role in generating SQL query predicates.

Metadata. We perform ablations on metadata. As per Table 9, column data types have a minor performance impact, possibly because the model infers types from column names and comments. Removing comments notably affects performance on the BIRD benchmark due to its ambiguous schemas. Additionally, both representative database values and primary/foreign keys are essential for performance on Spider and BIRD. The first offers insight into value format, and the latter helps in accurate `JOIN ON` clause generation.

9.6 Evaluation on Real-World Scenarios

We evaluate CodeS on two new domain datasets: Bank-Financials and Aminer-Simplified. The primary challenge of Bank-Financials lies in the large number of columns in the database and the presence of ambiguous schema names. Aminer-Simplified poses a challenge due to its complex and intricate table-join relationships.

For real-world deployment, we use the CodeS-7B model due to its balance between performance and efficiency. We use the schema item classifier trained on BIRD during inference to filter schemas in new databases, as it scores based on question-schema semantics and is adaptable across domains. Our baselines come from the prompting-based GPT-3.5. We provide GPT-3.5 with three random text-to-SQL samples from new databases. The input structure is: "[DDL] + [Comments (optional)] + 3 instances of [Question, SQL] + [Test question]". Given the EX metric's strictness, we also employ human evaluation (HE) for SQL query accuracy. For example, consider the question, "`What is the abstract of 'Optical geometries'?`". The human annotated ground truth SQL query is "`SELECT abstract FROM Paper WHERE title = 'Optical geometries';`". However, if the generated SQL query additionally selects the "`title`" column from the table, EX would judge it incorrect. Yet, human experts would consider the predicted SQL as valid since it provides the essential information requested by the user.

Considering available labeled data and computational resources, we offer the following pathways for using CodeS:

(1) For new databases without annotations, we can directly use the checkpoints fine-tuned on Spider and BIRD benchmarks for inference. Table 10 shows that such "SFT CodeS-7B using Spider" and "SFT CodeS-7B using BIRD w/ EK" have certain domain transfer capability. It should be noted that the large gap between EX and HE is attributed to the different annotation habits between benchmarks and our new datasets.

(2) With limited annotated samples, if computational resources are scarce, CodeS's few-shot learning can quickly adapt to new databases without parameter tuning. In our tests, CodeS-7B, using just three context demonstrations, could generate SQL queries for new databases (refer to "3-shot CodeS-7B" in Table 10). If resources permit, we can use the bi-directional data augmentation strategy proposed in Section 7 to produce ample training pairs for two new databases. Table 10's "SFT CodeS-7B using aug. data" shows that using these augmented data for fine-tuning CodeS-7B can greatly boost accuracy. However, fine-tuning a separate model for each database has substantial real-world overheads. We explored merging training data from Spider, BIRD, Bank-Financials, and Aminer-Simplified to train a unified text-to-SQL model. Results show that this approach not only prevents performance drops but

Table 8: Evaluation of SFT CodeS on Dr.Spider in terms of EX (%). "DB", "NLQ", and 'SQL" denote perturbations in the database, user's questions, and SQL side respectively. Macro-average is computed across various perturbations.

Type	Perturbation	#Samples	SMBOP [56]	RESDSQL-3B +NatSQL [36]	ChatGPT +ZeroNL2SQL [25]	SFT CodeS-1B	SFT CodeS-3B	SFT CodeS-7B	SFT CodeS-15B
DB	schema-synonym	2,619	53.9	68.3	**69.8**	58.5	64.3	67.2	66.9
	schema-abbreviation	2,853	59.0	70.0	74.8	68.6	75.0	76.8	**78.7**
	DBcontent-equivalence	382	37.2	40.1	**56.8**	53.9	47.9	46.9	47.6
	Average	-	50.0	59.4	**67.1**	60.3	62.4	63.6	64.4
NLQ	keyword-synonym	953	64.3	72.4	**74.0**	60.9	70.9	73.0	73.5
	keyword-carrier	399	79.2	83.5	88.2	86.5	91.2	91.5	**91.7**
	column-synonym	563	48.7	63.1	62.7	56.0	60.0	63.2	**64.7**
	column-carrier	579	64.6	63.9	71.7	67.4	74.4	**80.7**	79.1
	column-attribute	119	58.0	**71.4**	70.6	47.9	67.2	63.0	68.9
	column-value	304	58.9	**76.6**	76.0	72.4	75.0	73.7	76.3
	value-synonym	506	29.1	53.2	70.6	59.7	67.0	**72.7**	71.9
	multitype	1,351	46.1	60.7	66.4	57.5	66.5	**69.5**	69.4
	others	2,819	73.7	79.0	79.4	74.9	78.5	**81.5**	81.2
	Average	-	58.1	69.3	73.2	64.8	72.3	74.3	**75.2**
SQL	comparison	178	65.2	**82.0**	73.6	60.7	69.7	77.5	71.9
	sort-order	192	76.6	**85.4**	80.2	69.8	79.2	81.8	84.9
	nonDB-number	131	71.8	85.5	**92.4**	84.7	87.8	90.1	84.0
	DB-text	911	63.1	74.3	**80.7**	67.1	77.2	80.5	**80.7**
	DB-number	410	84.4	**88.8**	86.1	80.5	85.1	84.9	85.9
	Average	-	72.2	**83.2**	82.6	72.6	79.8	83.0	81.5
All	Global average	-	60.8	71.7	74.9	66.3	72.8	75.0	**75.1**

Table 9: Ablation studies on Spider's and BIRD's dev sets in the 3-shot in-context learning manner.

	Spider's dev (TS%)				BIRD's dev (EX%)			
	CodeS-1B	CodeS-3B	CodeS-7B	CodeS-15B	CodeS-1B	CodeS-3B	CodeS-7B	CodeS-15B
Original	57.4	67.8	69.8	71.5	25.42	31.23	33.44	35.14
Ablations on demonstration retriever								
-w/o pattern similarity	55.8(↓-1.6)	66.2(↓-1.6)	67.6(↓-2.2)	69.7(↓-1.8)	25.62(↑+0.20)	31.16(↓-0.07)	34.09(↑+0.65)	35.79(↑+0.65)
-w/o demonstration retriever	56.1(↓-1.3)	66.8(↓-1.0)	69.6(↓-0.2)	71.4(↓-0.1)	24.25(↓-1.17)	30.18(↓-1.05)	32.86(↓-0.58)	35.53(↑+0.39)
Ablations on schema filter and value retriever								
-w/o schema filter	55.0(↓-2.4)	65.4(↓-2.4)	69.0(↓-0.8)	70.2(↓-1.3)	23.53(↓-1.89)	30.64(↓-0.59)	33.83(↑+0.39)	33.57(↓-1.57)
-w/o value retriever	57.2(↓-0.2)	66.7(↓-1.1)	69.6(↓-0.2)	71.1(↓-0.4)	22.23(↓-3.19)	29.27(↓-1.96)	30.96(↓-2.48)	33.31(↓-1.83)
Ablations on metadata								
-w/o column data types	56.3(↓-1.1)	66.9(↓-0.9)	69.4(↓-0.4)	71.1(↓-0.4)	24.71(↓-0.71)	30.83(↓-0.4)	33.83(↑+0.39)	35.66(↑+0.52)
-w/o comments	57.7(↑+0.3)	67.0(↓-0.8)	69.2(↓-0.6)	71.0(↓-0.5)	24.71(↓-0.71)	29.92(↓-1.31)	32.33(↓-1.11)	34.03(↓-1.11)
-w/o representative values	57.6(↑+0.2)	66.4(↓-1.4)	69.9(↑+0.1)	70.4(↓-1.1)	23.92(↓-1.50)	29.40(↓-1.83)	30.77(↓-2.67)	31.94(↓-3.20)
-w/o primary and foreign keys	57.6(↑+0.2)	66.2(↓-1.6)	69.0(↓-0.8)	70.0(↓-1.5)	23.27(↓-2.15)	28.29(↓-2.94)	29.92(↓-3.52)	32.14(↓-3.00)

also improves performance, especially on the Aminer-Simplified dataset, as seen in Table 10's "SFT CodeS-7B using merged data".

9.7 Latency and Deployment Requirements

One of the key benefits of smaller models is their enhanced inference speed. To illustrate, the prior SOTA prompting-based method, DIN-SQL+GPT-4, reports approximately 60 seconds of inference time per sample on the Spider dataset. In contrast, our SFT CodeS demonstrates significantly improved efficiency. The inference times for the 1B, 3B, 7B, and 15B variants are only 0.6, 0.9, 1.1, and 1.5 seconds respectively on the same dataset. This highlights the superior speed of our models. In addition to its efficiency, CodeS is also suitable for real-world deployment. More specifically, when operating in float16 precision, the SFT CodeS variants (1B, 3B, 7B, 15B) require GPU memory capacities of 10GB, 13GB, 20GB, and 35GB, respectively. Therefore, we can deploy CodeS on local machines, bypassing the need for the expensive GPT-4 API.

10 CONLUSION

In this study, we have taken significant strides toward enhancing the landscape of open-source text-to-SQL models. With the

Table 10: Automatic and human evaluation results on Bank-Financials and Aminer-Simplified.

	Bank-Financials		Aminer-Simplified	
Methods	EX%	HE%	EX%	HE%
RESDSQL-3B + NatSQL	6.6	26.4	17.5	24.7
3-shot GPT-3.5	52.7	72.5	50.5	63.9
3-shot GPT-3.5 + comments	57.1	84.6	51.5	62.8
DIN-SQL + GPT-4	26.4	79.1	50.5	**67.0**
SFT CodeS-7B using Spider	11.0	73.6	27.8	36.1
SFT CodeS-7B using BIRD w/ EK	12.1	79.1	34.0	41.2
3-shot CodeS-7B	61.5	78.0	43.3	51.5
SFT CodeS-7B using aug. data	**71.4**	**85.7**	51.5	64.9
SFT CodeS-7B using merged data	65.9	84.6	**53.6**	**67.0**

introduction of CodeS, developers now have access to a range of specialized pre-trained language models to develop their text-to-SQL applications. We also open-source our collected SQL-focused corpus to the research community, which could pave the way for future innovations in SQL generation using incremental pre-training. In addition, we propose a comprehensive database prompt construction strategy and a novel bi-directional data augmentation method. This ensures that the model remains versatile and can adapt seamlessly to various domains. Finally, we conduct extensive evaluations across various text-to-SQL benchmarks. Our findings showcase that CodeS is the new SOTA pre-trained language model in the SQL generation capability and our SFT CodeS models achieve new SOTA accuracy and robustness on a wide range of text-to-SQL benchmarks. We hope our efforts, combined with the open source of our code, models, and data, will catalyze further exploration, adoption, and innovation in the domain of text-to-SQL. Beyond this, we're optimistic that this work could offer invaluable perspectives for deploying language models across other specialized domains.

REFERENCES

[1] Rohan Anil, Andrew M. Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, and et al. 2023. PaLM 2 Technical Report. *CoRR* abs/2305.10403 (2023). arXiv:2305.10403

[2] Ben Bogin, Matt Gardner, and Jonathan Berant. 2019. Global Reasoning over Database Structures for Text-to-SQL Parsing. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019*. 3657–3662.

[3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and et al. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*.

[4] Ursin Brunner and Kurt Stockinger. 2021. ValueNet: A Natural Language-to-SQL System that Learns from Database Information. In *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*. 2177–2182.

[5] Ruisheng Cao, Lu Chen, Zhi Chen, Yanbin Zhao, Su Zhu, and Kai Yu. 2021. LGESQL: Line Graph Enhanced Text-to-SQL Model with Mixed Local and Non-Local Relations. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021, (Volume 1: Long Papers), Virtual Event, August 1-6, 2021*. 2541–2555.

[6] Shuaichen Chang and Eric Fosler-Lussier. 2023. How to Prompt LLMs for Text-to-SQL: A Study in Zero-shot, Single-domain, and Cross-domain Settings. *CoRR* abs/2305.11853 (2023). arXiv:2305.11853

[7] Shuaichen Chang, Jun Wang, Mingwen Dong, Lin Pan, Henghui Zhu, Alexander Hanbo Li, Wuwei Lan, Sheng Zhang, Jiarong Jiang, Joseph Lilien, and et al. 2023. Dr.Spider: A Diagnostic Evaluation Benchmark towards Text-to-SQL Robustness. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*.

[8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and et al. 2021. Evaluating Large Language Models Trained on Code. *CoRR* abs/2107.03374 (2021). arXiv:2107.03374

[9] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching Large Language Models to Self-Debug. *CoRR* abs/2304.05128 (2023). arXiv:2304.05128

[10] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, and et al. 2022. PaLM: Scaling Language Modeling with Pathways. *CoRR* abs/2204.02311 (2022). arXiv:2204.02311

[11] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *CoRR* abs/2307.08691 (2023). arXiv:2307.08691

[12] Xiang Deng, Ahmed Hassan Awadallah, Christopher Meek, Oleksandr Polozov, Huan Sun, and Matthew Richardson. 2021. Structure-Grounded Pretraining for Text-to-SQL. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021*. 1337–1350.

[13] Xiang Deng, Ahmed Hassan Awadallah, Christopher Meek, Oleksandr Polozov, Huan Sun, and Matthew Richardson. 2021. Structure-Grounded Pretraining for Text-to-SQL. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021*. 1337–1350.

[14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*. 4171–4186.

[15] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. 2023. Enhancing Chat Language Models by Scaling High-quality Instructional Conversations. *CoRR* abs/2305.14233 (2023). arXiv:2305.14233

[16] Xuemei Dong, Chao Zhang, Yuhang Ge, Yuren Mao, Yunjun Gao, Lu Chen, Jinshu Lin, and Dongfang Lou. 2023. C3: Zero-shot Text-to-SQL with ChatGPT. *CoRR* abs/2307.07306 (2023). arXiv:2307.07306

[17] Longxu Dou, Yan Gao, Mingyang Pan, Dingzirui Wang, Jian-Guang Lou, Wanxiang Che, and Dechen Zhan. 2022. UniSAr: A Unified Structure-Aware Autoregressive Language Model for Text-to-SQL. *CoRR* abs/2203.07781 (2022). arXiv:2203.07781

[18] Han Fu, Chang Liu, Bin Wu, Feifei Li, Jian Tan, and Jianling Sun. 2023. CatSQL: Towards Real World Natural Language to SQL Applications. *Proc. VLDB Endow.* 16, 6 (2023), 1534–1547.

[19] Yujian Gan, Xinyun Chen, Qiuping Huang, Matthew Purver, John R. Woodward, Jinxia Xie, and Pengsheng Huang. 2021. Towards Robustness of Text-to-SQL Models against Synonym Substitution. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021, (Volume 1: Long Papers), Virtual Event, August 1-6, 2021*. 2505–2515.

[20] Yujian Gan, Xinyun Chen, and Matthew Purver. 2021. Exploring Underexplored Limitations of Cross-Domain Text-to-SQL Generalization. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*. 8926–8931.

[21] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2023. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *CoRR* abs/2308.15363 (2023). arXiv:2308.15363

[22] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, and et al. 2021. The Pile: An 800GB Dataset of Diverse Text for Language Modeling. *CoRR* abs/2101.00027 (2021). arXiv:2101.00027

[23] Tianyu Gao, Xingcheng Yao, and Danqi Chen. 2021. SimCSE: Simple Contrastive Learning of Sentence Embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*. 6894–6910.

[24] Zihui Gu, Ju Fan, Nan Tang, Lei Cao, Bowen Jia, Sam Madden, and Xiaoyong Du. 2023. Few-shot Text-to-SQL Translation using Structure and Content Prompt Learning. *Proc. ACM Manag. Data* 1, 2 (2023), 147:1–147:28.

[25] Zihui Gu, Ju Fan, Nan Tang, Songyue Zhang, Yuxin Zhang, Zui Chen, Lei Cao, Guoliang Li, Sam Madden, and Xiaoyong Du. 2023. Interleaving Pre-Trained Language Models and Large Language Models for Zero-Shot NL2SQL Generation. *CoRR* abs/2306.08891 (2023). arXiv:2306.08891

[26] Jiaqi Guo, Zecheng Zhan, Yan Gao, Yan Xiao, Jian-Guang Lou, Ting Liu, and Dongmei Zhang. 2019. Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation. In *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*. 4524–4535.

[27] Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Martin Eisenschlos. 2020. TaPas: Weakly Supervised Table Parsing via Pre-training. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*. 4320–4333.

[28] Or Honovich, Thomas Scialom, Omer Levy, and Timo Schick. 2023. Unnatural Instructions: Tuning Language Models with (Almost) No Human Labor. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023*. 14409–14428.

[29] Yiqun Hu, Yiyun Zhao, Jiarong Jiang, Wuwei Lan, Henghui Zhu, Anuj Chauhan, Alexander Hanbo Li, Lin Pan, Jun Wang, Chung-Wei Hang, and et al. 2023. Importance of Synthesizing High-quality Data for Text-to-SQL Parsing. In *Findings of the Association for Computational Linguistics: ACL 2023, Toronto, Canada, July 9-14, 2023*. 1327–1343.

[30] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, Jayant Krishnamurthy, and Luke Zettlemoyer. 2017. Learning a Neural Semantic Parser from User Feedback. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, Canada, July 30 - August 4, Volume 1: Long Papers*. 963–973.

[31] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, and et al. 2022. The Stack: 3 TB of permissively licensed source code. *CoRR* abs/2211.15533 (2022). arXiv:2211.15533

[32] Tiffany H. Kung, Morgan Cheatham, Arielle Medenilla, Czarina Sillos, Lorie De Leon, Camille Elepaño, Maria Madriaga, Rimel Aggabao, Giezel Diaz-Candido, James Maningo, and et al. 2023. Performance of ChatGPT on USMLE: Potential for AI-assisted medical education using large language models. *PLOS Digital Health* 2, 2 (02 2023), 1–12.

[33] Wenqiang Lei, Weixin Wang, Zhixin Ma, Tian Gan, Wei Lu, Min-Yen Kan, and Tat-Seng Chua. 2020. Re-examining the Role of Schema Linking in Text-to-SQL. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020*. 6943–6954.

[34] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*. 7871–7880.

[35] Fei Li and H. V. Jagadish. 2014. Constructing an Interactive Natural Language Interface for Relational Databases. *Proc. VLDB Endow.* 8, 1 (2014), 73–84.

[36] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. 2023. RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL. In *Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023*. 13067–13075.

[37] Jinyang Li, Binyuan Hui, Reynold Cheng, Bowen Qin, Chenhao Ma, Nan Huo, Fei Huang, Wenyu Du, Luo Si, and Yongbin Li. 2023. Graphix-T5: Mixing Pre-trained Transformers with Graph-Aware Layers for Text-to-SQL Parsing. In *Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023*. 13076–13084.

[38] Jinyang Li, Binyuan Hui, Ge Qu, Binhua Li, Jiaxi Yang, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, and et al. 2023. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. *CoRR* abs/2305.03111 (2023). arXiv:2305.03111

[39] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, and et al. 2023. StarCoder: may the source be with you! *CoRR* abs/2305.06161 (2023). arXiv:2305.06161

[40] Aiwei Liu, Xuming Hu, Lijie Wen, and Philip S. Yu. 2023. A comprehensive evaluation of ChatGPT's zero-shot Text-to-SQL capability. *CoRR* abs/2303.13547 (2023). arXiv:2303.13547

[41] Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. 2022. What Makes Good In-Context Examples for GPT-3?. In *Proceedings of Deep Learning Inside Out: The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures, DeeLIO@ACL 2022, Dublin, Ireland and Online, May 27, 2022*. 100–114.

[42] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. *ACM Comput. Surv.* 55, 9 (2023), 195:1–195:35.

[43] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *CoRR* abs/1907.11692 (2019). arXiv:1907.11692

[44] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*.

[45] Erik Nijkamp, Hiroaki Hayashi, Caiming Xiong, Silvio Savarese, and Yingbo Zhou. 2023. CodeGen2: Lessons for Training LLMs on Programming and Natural Languages. *CoRR* abs/2305.02309 (2023). arXiv:2305.02309

[46] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*.

[47] Harsha Nori, Nicholas King, Scott Mayer McKinney, Dean Carignan, and Eric Horvitz. 2023. Capabilities of GPT-4 on Medical Challenge Problems. *CoRR* abs/2303.13375 (2023). arXiv:2303.13375

[48] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023). arXiv:2303.08774

[49] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, and et al. 2022. Training language models to follow instructions with human feedback. In *NeurIPS*.

[50] Ana-Maria Popescu, Oren Etzioni, and Henry A. Kautz. 2003. Towards a theory of natural language interfaces to databases. In *Proceedings of the 8th International Conference on Intelligent User Interfaces, IUI 2003, Miami, FL, USA, January 12-15, 2003*. 149–157.

[51] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. *CoRR* abs/2304.11015 (2023). arXiv:2304.11015

[52] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. (2018).

[53] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *J. Mach. Learn. Res.* 21 (2020), 140:1–140:67.

[54] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020*. 20.

[55] Nitarshan Rajkumar, Raymond Li, and Dzmitry Bahdanau. 2022. Evaluating the Text-to-SQL Capabilities of Large Language Models. *CoRR* abs/2204.00498 (2022). arXiv:2204.00498

[56] Ohad Rubin and Jonathan Berant. 2020. SmBoP: Semi-autoregressive Bottom-up Semantic Parsing. *CoRR* abs/2010.12412 (2020). arXiv:2010.12412

[57] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*. 9895–9901.

[58] Peter Shaw, Ming-Wei Chang, Panupong Pasupat, and Kristina Toutanova. 2021. Compositional Generalization and Natural Language Variation: Can a Semantic Parsing Approach Handle Both?. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021*. 922–938.

[59] Peng Shi, Patrick Ng, Zhiguo Wang, Henghui Zhu, Alexander Hanbo Li, Jun Wang, Cícero Nogueira dos Santos, and Bing Xiang. 2021. Learning Contextual Representations for Semantic Parsing with Generation-Augmented Pre-Training. In *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*. 13806–13814.

[60] Arnab Sinha, Zhihong Shen, Yang Song, Hao Ma, Darrin Eide, Bo-June Paul Hsu, and Kuansan Wang. 2015. An Overview of Microsoft Academic Service (MAS) and Applications. In *Proceedings of the 24th International Conference on World Wide Web Companion, WWW 2015, Florence, Italy, May 18-22, 2015 - Companion Volume*. 243–246.

[61] Ruoxi Sun, Sercan Ö. Arik, Hootan Nakhost, Hanjun Dai, Rajarishi Sinha, Pengcheng Yin, and Tomas Pfister. 2023. SQL-PaLM: Improved Large Language Model Adaptation for Text-to-SQL. *CoRR* abs/2306.00739 (2023). arXiv:2306.00739

[62] Jie Tang, Jing Zhang, Limin Yao, Juanzi Li, Li Zhang, and Zhong Su. 2008. ArnetMiner: extraction and mining of academic social networks. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Las Vegas, Nevada, USA, August 24-27, 2008*. 990–998.

[63] Lappoon R. Tang and Raymond J. Mooney. 2001. Using Multiple Clause Constructors in Inductive Logic Programming for Semantic Parsing. In *Machine Learning: EMCL 2001, 12th European Conference on Machine Learning, Freiburg, Germany, September 5-7, 2001, Proceedings (Lecture Notes in Computer Science, Vol. 2167)*. 466–477.

[64] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, and et al. 2023. LLaMA: Open and Efficient Foundation Language Models. *CoRR* abs/2302.13971 (2023). arXiv:2302.13971

[65] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, and et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *CoRR* abs/2307.09288 (2023). arXiv:2307.09288

[66] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*. 5998–6008.

[67] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2020. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*. 7567–7578.

[68] Bailin Wang, Wenpeng Yin, Xi Victoria Lin, and Caiming Xiong. 2021. Learning to Synthesize Data for Semantic Parsing. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021*. 2760–2766.

[69] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. Self-Instruct: Aligning Language Models with Self-Generated Instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023*. 13484–13508.

[70] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *NeurIPS*.

[71] Kun Wu, Lijie Wang, Zhenghua Li, Ao Zhang, Xinyan Xiao, Hua Wu, Min Zhang, and Haifeng Wang. 2021. Data Augmentation with Hierarchical SQL-to-Question Generation for Cross-domain Text-to-SQL Parsing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*. 8974–8983.

[72] Navid Yaghmazadeh, Yuepeng Wang, Isil Dillig, and Thomas Dillig. 2017. SQLizer: query synthesis from natural language. *Proc. ACM Program. Lang.* 1, OOPSLA (2017), 63:1–63:26.

[73] Ziyu Yao, Daniel S. Weld, Wei-Peng Chen, and Huan Sun. 2018. StaQC: A Systematically Mined Question-Code Dataset from Stack Overflow. In *Proceedings of the 2018 World Wide Web Conference on World Wide Web, WWW 2018, Lyon, France, April 23-27, 2018*. 1693–1703.

[74] Pengcheng Yin, Bowen Deng, Edgar Chen, Bogdan Vasilescu, and Graham Neubig. 2018. Learning to mine aligned code and natural language pairs from stack overflow. In *Proceedings of the 15th International Conference on Mining Software Repositories, MSR 2018, Gothenburg, Sweden, May 28-29, 2018*. 476–486.

[75] Pengcheng Yin and Graham Neubig. 2017. A Syntactic Neural Model for General-Purpose Code Generation. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, Canada, July 30 - August 4, Volume 1: Long Papers*. 440–450.

[76] Pengcheng Yin, Graham Neubig, Wen-tau Yih, and Sebastian Riedel. 2020. TaBERT: Pretraining for Joint Understanding of Textual and Tabular Data. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*. 8413–8426.

[77] Tao Yu, Chien-Sheng Wu, Xi Victoria Lin, Bailin Wang, Yi Chern Tan, Xinyi Yang, Dragomir R. Radev, Richard Socher, and Caiming Xiong. 2021. GraPPa: Grammar-Augmented Pre-Training for Table Semantic Parsing. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*.

[78] Tao Yu, Michihiro Yasunaga, Kai Yang, Rui Zhang, Dongxu Wang, Zifan Li, and Dragomir R. Radev. 2018. SyntaxSQLNet: Syntax Tree Networks for Complex and Cross-Domain Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*. 1653–1663.

[79] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, and et al. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*. 3911–3921.

[80] John M. Zelle and Raymond J. Mooney. 1996. Learning to Parse Database Queries Using Inductive Logic Programming. In *Proceedings of the Thirteenth National Conference on Artificial Intelligence and Eighth Innovative Applications of Artificial Intelligence Conference, AAAI 96, IAAI 96, Portland, Oregon, USA, August 4-8, 1996, Volume 2*. 1050–1055.

[81] Fanjin Zhang, Xiao Liu, Jie Tang, Yuxiao Dong, Peiran Yao, Jie Zhang, Xiaotao Gu, Yan Wang, Evgeny Kharlamov, Bin Shao, and et al. 2023. OAG: Linking Entities Across Large-Scale Heterogeneous Knowledge Graphs. *IEEE Trans. Knowl. Data Eng.* 35, 9 (2023), 9225–9239.

[82] Fanjin Zhang, Xiao Liu, Jie Tang, Yuxiao Dong, Peiran Yao, Jie Zhang, Xiaotao Gu, Yan Wang, Bin Shao, Rui Li, and et al. 2019. OAG: Toward Linking Large-scale Heterogeneous Entity Graphs. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2019, Anchorage, AK, USA, August 4-8, 2019*. 2585–2595.

[83] Yi Zhang, Jan Deriu, George Katsogiannis-Meimarakis, Catherine Kosten, Georgia Koutrika, and Kurt Stockinger. 2023. ScienceBenchmark: A Complex Real-World Benchmark for Evaluating Natural Language to SQL Systems. *CoRR* abs/2306.04743 (2023). arXiv:2306.04743

[84] Yiming Zhang, Shi Feng, and Chenhao Tan. 2022. Active Example Selection for In-Context Learning. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*. 9134–9148.

[85] Ruiqi Zhong, Tao Yu, and Dan Klein. 2020. Semantic Evaluation for Text-to-SQL with Distilled Test Suites. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020*. 396–411.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009